

Dosificador de Bebedero

Documentacion del Proyecto y API REST

Sistema IoT para monitoreo y dosificacion automatica
de bebederos de agua con analisis de imagenes

Descripcion del Proyecto

Sistema IoT desarrollado en Python con FastAPI para el monitoreo y dosificacion automatica de un bebedero de agua. El sistema utiliza un sensor ultrasonico para medir el nivel de agua, una camara para capturar fotos del bebedero y un analizador de imagenes para calcular el porcentaje de cobertura de capsulas en la superficie del agua.

Si la cobertura detectada esta por debajo del objetivo configurado, el sistema inicia automaticamente el proceso de dosificacion hasta alcanzar el nivel deseado.

Arquitectura

El proyecto esta organizado en los siguientes modulos:

api_main.py	- Entry point de la API REST (FastAPI)
main.py	- Workflow principal de dosificacion automatica
core/	- Logica de hardware: sensor, camara, analizador, dosificador
api/	- API REST: routes, controllers, models
utilities/	- Helpers: scheduler, file_handler, logging
error_handling/	- Manejo de errores y troubleshooting
send_info/	- Envio de informacion y alertas

Endpoints de la API

Configuracion

GET	/config	Obtiene la configuracion completa
PATCH	/basic-config	Actualiza configuraciones basicas
PATCH	/config	Actualiza configuracion (campos nuevos)

Ejemplo - Obtener configuracion:

```
curl http://localhost:8000/config
```

Ejemplo - Actualizar configuracion basica:

```
curl -X PATCH http://localhost:8000/basic-config \
-H "Content-Type: application/json" \
-d '{
  "watertank": {"coverage": 85},
  "set_active": true,
  "reschedule_hours": {"normal": 24, "error": 2}
}'
```

Ejemplo - Actualizar configuracion arbitraria:

```
curl -X PATCH http://localhost:8000/config \
-H "Content-Type: application/json" \
-d '{
  "watertank": {"coverage": 90},
  "nuevo_campo": "valor"
}'
```

Camara

POST	/camera/capture	Toma una foto y devuelve base64
GET	/camera/last-image	Obtiene la ultima imagen tomada
GET	/camera/photo/{nombre}	Obtiene una imagen especifica

Ejemplo - Capturar foto:

```
curl -X POST http://localhost:8000/camera/capture
# Respuesta: {"nombre": "foto_manual.jpg", "imagen": "base64..."}
```

Ejemplo - Obtener ultima imagen:

```
curl http://localhost:8000/camera/last-image
```

Ejemplo - Obtener foto especifica:

```
curl http://localhost:8000/camera/photo/ultima_foto.jpg
```

Sensores

GET	/water-status	Estado del agua (distancia y nivel)
GET	/health	Estado de salud de componentes

Ejemplo - Estado del agua:

```
curl http://localhost:8000/water-status
# Respuesta: {"distancia": 50, "nivel_de_agua": "50/100", "error": null}
```

Ejemplo - Health check:

```
curl http://localhost:8000/health
# Respuesta: {
#   "ultrasonic_sensor": true,
#   "camera": true,
#   "analyzer": true,
#   "config": true,
#   "log": true
# }
```

Analisis de Imagenes

GET	/analyze/image/{nombre}	Analiza una imagen especifica
GET	/analyze/last-image	Analiza la ultima imagen tomada

Ejemplo - Analizar imagen especifica:

```
curl http://localhost:8000/analyze/image/ultima_foto.jpg
# Respuesta: {"resultado": 0.75, "nombre": "ultima_foto.jpg"}
```

Ejemplo - Analizar ultima imagen:

```
curl http://localhost:8000/analyze/last-image
```

Logs

GET	/log/full	Log completo con imagenes en base64
GET	/log/text	Solo texto del log
GET	/log/photos	Descarga imagenes del log en ZIP

Ejemplo - Log completo:

```
curl http://localhost:8000/log/full
```

Ejemplo - Solo texto del log:

```
curl http://localhost:8000/log/text
# Respuesta: {"log": "2025-03-12 18:00:00 - INFO - ..."} 
```

Ejemplo - Descargar imagenes como ZIP:

```
curl -o log_imagenes.zip http://localhost:8000/log/photos
```

Scheduler

POST	/scheduler/clear-jobs	Elimina tareas programadas
POST	/scheduler/run-job	Programa ejecucion de un script
GET	/scheduler/list-jobs	Lista tareas pendientes

Ejemplo - Limpiar tareas programadas:

```
curl -X POST http://localhost:8000/scheduler/clear-jobs
# Respuesta: {"message": "Todas las tareas programadas han sido eliminadas."}
```

Ejemplo - Programar ejecucion de script:

```
curl -X POST http://localhost:8000/scheduler/run-job \
-H "Content-Type: application/json" \
-d '{"script": "main.py", "minutes": 30}'
# Respuesta: {"message": "Se ha programado "main.py" para ejecutarse en 30 minutos."}
```

Ejemplo - Listar tareas pendientes:

```
curl http://localhost:8000/scheduler/list-jobs
# Respuesta: {"lista": [...]}
```

Configuracion por Defecto (config.yml)

```
camera:
  total_photo: 3          # Fotos por analisis
reschedule_hours:
  error: 1                # Reintentar tras error (horas)
  normal: 24              # Reintentar normal (horas)
set_active: true          # Sistema activado
ultrasound:
  empty: 100              # Distancia cuando esta vacio
  full: 0                 # Distancia cuando esta lleno
volumetric:
  deep: 100               # Profundidad del bebedero
  height: 100             # Altura
  length: 200             # Largo
watertank:
  coverage: 80            # Cobertura objetivo (%)
```

Como iniciar la API

```
cd dosificador
uvicorn api_main:app --host 0.0.0.0 --port 8000
```

Una vez iniciada, la documentacion interactiva de Swagger estara disponible en:
<http://localhost:8000/docs>